Additional Premium Features

Workspace Persistence

The extension should remember the complete workspace state for every problem.

When reopening a previously visited problem, restore:

- Cursor position
- Scroll position
- Selected programming language
- Editor contents (if not submitted)
- Split-screen ratio
- Open sidebar panels
- Active notes
- Open console
- Selected test case
- Font size
- Theme
- Zoom level
- Undo and redo history
- Folded code regions
- Last active workspace profile

Users should be able to continue exactly where they left off.

Command Palette

Provide a universal command palette similar to modern development environments.

The command palette should allow users to instantly access every major feature without navigating menus.

Examples include:

- Run Code
- Submit Solution
- Open Notes
- Search Problems
- Toggle Theme
- Change Language
- Switch Workspace

- Bookmark Problem
- Copy Sample Input
- Copy Sample Output
- Open Settings
- Toggle Zen Mode
- Toggle Sidebar
- Export Workspace
- Import Workspace

The command palette should support fuzzy search and keyboard navigation.

Workspace Profiles

Support multiple workspace configurations.

Example profiles include:

- Practice
- Contest
- Interview Preparation
- Teaching
- Minimal
- Custom

Each profile should remember:

- Layout
- Theme
- Visible panels
- Keyboard shortcuts
- Font size
- Console placement
- Sidebar configuration

Zen Mode

Zen Mode removes every unnecessary interface element.

Hide:

- Navigation bar
- Footer
- Rating

- Discussions
- Submission history
- Sidebar
- Extra panels

Display only:

- Problem Statement
- Editor
- Console

Designed for distraction-free solving.

Focus Mode

Focus Mode is intended specifically for contests and uninterrupted practice.

Hide:

- Editorial links
- Discussion links
- Rating indicators
- Solved statistics
- Tags
- Recommendations
- Analytics

Maintain a clean environment centered on solving the current problem.

Integrated Console

Replace the default execution experience with a fully integrated console.

Features include:

- Compile status
- Runtime output
- Expected output
- Difference viewer
- Standard input editor
- Multiple test execution
- Runtime information
- Memory usage
- Error highlighting

- Compiler messages
- Execution history

Intelligent Problem Navigation

Generate a structured navigation panel for every problem.

Automatically detect sections including:

- Description
- Input
- Output
- Constraints
- Examples
- Notes

Support:

- Instant scrolling
- Active section highlighting
- Keyboard navigation
- Collapsible sections

Advanced Markdown Notes

Notes should support rich formatting.

Supported features include:

- Headers
- Tables
- Checklists
- Code blocks
- Inline code
- Mathematical expressions
- Images
- Hyperlinks
- Blockquotes

Users should be able to pin important observations.

Quick Clipboard Actions

Provide one-click copying for:

- Sample Input
- Sample Output
- Expected Output
- Entire Problem Statement
- Code
- Problem URL
- Problem Title
- Constraints

Smart Settings

Settings should include a searchable interface.

Users should be able to instantly locate any configurable option using keywords.

Notification Center

Replace browser alerts with a modern notification system.

Notifications include:

- Workspace Saved
- Code Submitted
- Execution Completed
- Accepted
- Wrong Answer
- Compilation Failed
- Settings Updated
- Backup Completed

Notifications should be animated, lightweight, and non-intrusive.

Skeleton Loading

Display animated loading placeholders while content loads.

Avoid blank screens and layout shifts.

Maintain a polished visual experience during every loading state.

Micro Interactions

Every interaction should feel responsive.

Examples include:

- Button animations
- Hover effects
- Sidebar transitions
- Card animations
- Console transitions
- Verdict animations
- Panel resizing
- Smooth scrolling
- Context menus

Animations should remain subtle and never interfere with productivity.

Refined Empty States

Every empty screen should guide the user.

Examples include:

- No Bookmarks
- No Notes
- No Recent Problems
- No Custom Tests

Include meaningful illustrations and helpful actions.

Theme Builder

Allow users to create completely custom themes.

Customization includes:

- Accent colors
- Background colors
- Typography
- Border radius
- Shadow intensity
- Blur intensity
- Transparency
- Animation speed

Users should be able to import and export themes.

Layout Presets

Provide professionally designed workspace layouts.

Examples include:

- Classic
- Competitive
- Minimal
- LeetCode Inspired
- VS Code Inspired
- Wide Editor
- Reading Mode

Layouts should remain fully customizable.

Offline Caching

Cache previously visited problems locally.

Enable:

- Faster reopening
- Instant rendering
- Reduced network dependency
- Cached examples
- Cached metadata

Automatically refresh cached content when newer versions are available.

Recent Activity

Maintain a history of user activity.

Examples include:

- Recently opened problems
- Recently viewed contests
- Recently used commands
- Recently edited notes

Allow users to clear history at any time.

Advanced Search

Support global search across:

- Problem titles
- Contest names
- Tags
- Difficulty
- Bookmarks
- Notes
- Recently opened problems

Provide instant search results with keyboard navigation.

Practice Analytics

Provide detailed analytics including:

- Total problems solved
- Problems attempted
- Acceptance rate
- Average solving time
- Difficulty distribution
- Language usage
- Topic mastery
- Weakest topics
- Strongest topics
- Rating progression
- Contest history

- Longest solving streak
- Daily activity
- Weekly activity
- Monthly activity

Visualize progress using charts and a contribution-style heatmap.

Achievement System

Provide optional milestones to motivate consistent practice.

Examples include:

- First Accepted Solution
- 100 Problems Solved
- 500 Problems Solved
- 30-Day Streak
- First Contest
- First Specialist
- First Expert
- First Candidate Master
- Topic Mastery Milestones

Users should be able to disable achievements if preferred.

Practice Timer

Provide an integrated timer for focused solving.

Support:

- Stopwatch
- Countdown
- Pomodoro sessions
- Session history
- Auto-start on opening a problem

Backup and Restore

Allow users to export and import their local data.

Exportable data includes:

- Settings
- Notes
- Bookmarks
- Themes
- Workspace Profiles
- Practice History
- Custom Test Cases

Support secure JSON backups for easy migration between devices.

Crash Recovery

Automatically recover unsaved work after unexpected interruptions.

Recover:

- Editor contents
- Console state
- Notes
- Workspace layout
- Cursor position
- Active test cases

No work should be lost due to accidental refreshes or browser crashes.

Advanced Tools

Provide optional utilities for advanced users.

Examples include:

- Storage Inspector
- Workspace Reset
- Performance Statistics
- Cache Management
- Debug Logs
- Extension Diagnostics

These tools should remain hidden unless explicitly enabled.

Product Identity

The product should establish its own visual language rather than replicating another platform.

Every interface element should communicate:

- Professionalism
- Speed
- Simplicity
- Precision
- Reliability
- Modern software craftsmanship

The goal is not to imitate existing competitive programming platforms, but to become the definitive premium workspace for solving Codeforces problems.

Every design decision should reinforce the feeling that the user is working inside a dedicated competitive programming IDE rather than a traditional website.